

LAPORAN PRAKTIKUM

Struktur Data dan Algoritma

Menggunakan JavaScript & Node.js

MODUL 4

Object, Class & OOP

Topik Utama: Object Literal, Destructuring, Spread/Rest,

Class, Constructor, Method, Inheritance, this

Tools: Node.js v18+, VS Code, Terminal

Dokumen: Modul Praktikum — Teori, Kode, Latihan & Tugas

Sumber Modul: Muhammad Reza Zulman, M.Sc

2026 All right reserved.

Daftar Isi

BAB 1 — Pendahuluan

- 1.1 Informasi Modul
- 1.2 Tujuan Pembelajaran
- 1.3 Setup Project

BAB 2 — Object Literal

- 2.1 Teori: Apa itu Object?
- 2.2 Praktik: Membuat Object Literal
- 2.3 Latihan 1: Object Buku dan Perpustakaan

BAB 3 — Destructuring, Spread & Rest

- 3.1 Teori: Sintaks Modern ES6+
- 3.2 Praktik: Destructuring, Spread, dan Rest
- 3.3 Latihan 2: Pengolahan Data

BAB 4 — Class

- 4.1 Teori: OOP dengan Class
- 4.2 Praktik: Mendefinisikan Class
- 4.3 Latihan 3: Class BankAccount

BAB 5 — Inheritance (Pewarisan)

- 5.1 Teori: Pewarisan Class
- 5.2 Praktik: Inheritance dan Polymorphism

5.3 Latihan 4: Hierarki Class Hewan

BAB 6 — Implementasi Stack dan Queue

6.1 Teori: Stack dan Queue

6.2 Praktik: Implementasi Stack dan Queue

BAB 7 — Tugas Praktikum

7.1 Tugas 1: Sistem Manajemen Toko Online

7.2 Tugas 2: Stack Kalkulator (Evaluasi Ekspresi)

BAB 8 — Kesimpulan

Laporan Praktikum Modul 4 — Object, Class & OOP

BAB 1 — Pendahuluan

1.1 Informasi Modul

Mata Kuliah	Struktur Data dan Algoritma
Modul	Modul 4
Topik Utama	Object, Class, dan OOP (Object-Oriented Programming)
Sub Topik	Object Literal, Destructuring, Spread & Rest, Class, Constructor, Method, Inheritance, this
Tools	Node.js v18+, VS Code, Terminal
Prasyarat	Menyelesaikan Modul Praktikum 2 dan 3

1.2 Tujuan Pembelajaran

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Membuat dan mengakses object literal beserta property dan method-nya.
2. Menggunakan destructuring untuk mengekstrak nilai dari object dan array.
3. Menggunakan spread operator (`...`) untuk menyalin dan menggabungkan object/array.
4. Menggunakan rest parameter untuk menangkap argumen dengan jumlah tidak tentu.
5. Mendefinisikan class dengan constructor, property, dan method.
6. Memahami konsep `this` dan bagaimana ia merujuk ke object saat ini.
7. Mengimplementasikan inheritance (pewarisan) menggunakan `extends` dan `super`.
8. Membuat implementasi Stack dan Queue sederhana menggunakan Class sebagai jembatan ke materi DSA.

1.3 Setup Project

Buat folder `praktikum-4/` di dalam repository GitHub dan inialisasi npm project:

Langkah 1: Navigasikan ke root folder repository
`cd nama-repository-kamu`

```
# Langkah 2: Buat folder praktikum-4/ dan masuk ke dalamnya
mkdir praktikum-4
cd praktikum-4
```

```
# Langkah 3: Inisialisasi npm project
npm init -y
```

Struktur Folder yang Diharapkan

```
nama-repository/
├── praktikum-4/
│   ├── package.json
│   ├── 01-object-literal.js
│   ├── 02-destructuring-spread.js
│   ├── 03-class-dasar.js
│   ├── 04-inheritance.js
│   ├── 05-stack-queue.js
│   └── tugas/
│       ├── tugas-1.js
│       └── tugas-2.js
```

Laporan Praktikum Modul 4 — Object, Class & OOP

BAB 2 — Object Literal

2.1 Teori: Apa itu Object?

Jika array menyimpan data secara terurut menggunakan indeks angka, **object** menyimpan data menggunakan *key-value pairs* (pasangan kunci-nilai). Object memungkinkan kita merepresentasikan entitas nyata dengan lebih deskriptif.

Array (kurang deskriptif)	<code>['Budi', 20, 'Informatika', 3.5]</code>	Harus mengingat posisi indeks
Object (lebih deskriptif)	<code>{ nama: 'Budi', umur: 20, jurusan: 'Informatika', ipk: 3.5 }</code>	Key menjelaskan makna data

Komponen Utama Object

- **Property** — pasangan key-value yang menyimpan data. Key adalah string (atau Symbol), value bisa berupa tipe data apapun.
- **Method** — property yang nilainya adalah sebuah function. Method mendefinisikan perilaku (behavior) dari object.
- `this` — kata kunci spesial di dalam method yang merujuk ke object itu sendiri tempat method berada.

Cara Mengakses Property

- `mahasiswa.nama` — *Dot notation*: cara paling umum dan direkomendasikan.
- `mahasiswa['nama']` — *Bracket notation*: diperlukan jika key berupa variabel atau mengandung spasi.

2.2 Praktik: Membuat Object Literal

Berikut adalah kode lengkap file `01-object-literal.js` yang mendemonstrasikan pembuatan object, pengaksesan property, penambahan/penghapusan property, penggunaan method dan `this`, serta iterasi object:

```
// =====  
// OBJECT LITERAL: PROPERTY, METHOD, DAN THIS  
// =====  
  
// --- 1. Membuat object literal ---  
const mahasiswa = {  
  nama : 'Budi Santoso',  
  umur : 20,  
  jurusan : 'Teknik Informatika',  
  ipk : 3.75,  
  aktif : true,  
};  
  
// --- 2. Mengakses property ---  
console.log('=== Akses Property ===');  
console.log('Nama :', mahasiswa.nama); // dot notation  
console.log('Jurusan :', mahasiswa['jurusan']); // bracket notation  
  
// Bracket notation berguna jika key disimpan di variabel  
const keyYangDicari = 'ipk';  
console.log('IPK :', mahasiswa[keyYangDicari]);  
  
// --- 3. Menambah dan mengubah property ---  
mahasiswa.email = 'budi@email.com'; // tambah property baru  
mahasiswa.umur = 21; // ubah nilai yang ada  
console.log('\nSetelah diubah:', mahasiswa);  
  
// --- 4. Menghapus property ---  
delete mahasiswa.aktif;  
console.log('Setelah delete:', mahasiswa);  
  
// --- 5. Menambahkan method ke object ---  
const dosen = {  
  nama : 'Dr. Ahmad Fauzi',  
  mataKuliah : 'Struktur Data',  
  pengalaman : 10,  
  
  perkenalan() {  
    // 'this' merujuk ke object dosen itu sendiri  
    return `Halo, saya ${this.nama}, mengajar ${this.mataKuliah}.`;  
  },  
  
  statusSenior() {  
    if (this.pengalaman >= 10) {  
      return `${this.nama} adalah dosen senior.`;  
    }  
    return `${this.nama} adalah dosen junior.`;  
  }  
};  
  
console.log('\n=== Method Object ===');  
console.log(dosen.perkenalan());  
console.log(dosen.statusSenior());  
  
// --- 6. Iterasi key-value dengan for...in ---  
console.log('\n=== Iterasi Object ===');  
for (const key in dosen) {
```

```

    if (typeof dosen[key] !== 'function') {
      console.log(` ${key}: ${dosen[key]}`);
    }
  }
}

```

```

console.log('Keys :', Object.keys(mahasiswa));
console.log('Values:', Object.values(mahasiswa));

```

Output yang Dihasilkan

```

=== Akses Property ===
Nama   : Budi Santoso
Jurusan : Teknik Informatika
IPK    : 3.75

Setelah diubah: { nama: 'Budi Santoso', umur: 21, jurusan: 'Teknik Informatika', ipk: 3.75, aktif: true, email: 'budi@email.com' }
Setelah delete: { nama: 'Budi Santoso', umur: 21, jurusan: 'Teknik Informatika', ipk: 3.75, email: 'budi@email.com' }

=== Method Object ===
Halo, saya Dr. Ahmad Fauzi, mengajar Struktur Data.
Dr. Ahmad Fauzi adalah dosen senior.

=== Iterasi Object ===
nama: Dr. Ahmad Fauzi
mataKuliah: Struktur Data
pengalaman: 10
Keys   : [ 'nama', 'umur', 'jurusan', 'ipk', 'email' ]
Values : [ 'Budi Santoso', 21, 'Teknik Informatika', 3.75, 'budi@email.com' ]

```

Analisis: Perhatikan bahwa `this.nama` di dalam method `perkenalan()` secara otomatis merujuk ke property `nama` milik object `dosen`. Setelah `delete mahasiswa.aktif`, property tersebut hilang dari object.

Laporan Praktikum Modul 4 — Object, Class & OOP

2.3 Latihan 1: Object Buku dan Perpustakaan

Berikut adalah kode penyelesaian Latihan 1 yang ditambahkan di bawah kode praktik sebelumnya pada file `01-object-literal.js`:

```

// =====
// LATIHAN 1: OBJECT BUKU DAN PERPUSTAKAAN
// =====

const buku = {
  judul      : 'Algoritma dan Pemrograman',
  pengarang  : 'Thomas Cormen',
  tahunTerbit : 2020,
  harga      : 150000,
  tersedia   : true,

  // Method info() — deskripsi lengkap buku
  info() {
    return `"${this.judul}" oleh ${this.pengarang} (${this.tahunTerbit}) — Rp${this.harga.toLocaleString()} | Tersedia: ${this.tersedia ? 'Ya' : 'Tidak'}";`
  },

  // Method diskon(persen) — menghitung harga setelah diskon
  diskon(persen) {
    const hargaDiskon = this.harga * (1 - persen / 100);
    return hargaDiskon;
  }
}

```

```

};

console.log("\n=== Info Buku ===");
console.log(buku.info());
console.log(`Harga setelah diskon 10%: Rp${buku.diskon(10).toLocaleString()}`);

// Array koleksiBuku berisi minimal 3 object buku
const koleksiBuku = [
  {
    judul: 'Algoritma dan Pemrograman',
    pengarang: 'Thomas Cormen',
    tahunTerbit: 2020,
    harga: 150000,
    tersedia: true,
    info() {
      return `${this.judul} — ${this.pengarang} (${this.tahunTerbit}) Rp${this.harga.toLocaleString()}`;
    }
  },
  {
    judul: 'Struktur Data Modern',
    pengarang: 'Adam Drozdek',
    tahunTerbit: 2022,
    harga: 180000,
    tersedia: false,
    info() {
      return `${this.judul} — ${this.pengarang} (${this.tahunTerbit}) Rp${this.harga.toLocaleString()}`;
    }
  },
  {
    judul: 'JavaScript: The Good Parts',
    pengarang: 'Douglas Crockford',
    tahunTerbit: 2019,
    harga: 95000,
    tersedia: true,
    info() {
      return `${this.judul} — ${this.pengarang} (${this.tahunTerbit}) Rp${this.harga.toLocaleString()}`;
    }
  }
];

// forEach: menampilkan info semua buku
console.log("\n=== Semua Buku di Koleksi ===");
koleksiBuku.forEach((b, i) => {
  console.log(`${i + 1}. ${b.info()}`);
});

// filter: mendapatkan buku yang tersedia
const bukuTersedia = koleksiBuku.filter(b => b.tersedia === true);
console.log("\n=== Buku yang Tersedia ===");
bukuTersedia.forEach((b, i) => {
  console.log(`${i + 1}. ${b.judul}`);
});

```

Output Latihan 1

=== Info Buku ===

"Algoritma dan Pemrograman" oleh Thomas Cormen (2020) — Rp150.000 | Tersedia: Ya
 Harga setelah diskon 10%: Rp135.000

=== Semua Buku di Koleksi ===

1. "Algoritma dan Pemrograman" — Thomas Cormen (2020) Rp150.000
2. "Struktur Data Modern" — Adam Drozdek (2022) Rp180.000
3. "JavaScript: The Good Parts" — Douglas Crockford (2019) Rp95.000

BAB 3 — Destructuring, Spread & Rest

3.1 Teori: Sintaks Modern ES6+

Destructuring adalah cara ringkas untuk mengekstrak nilai dari object atau array ke dalam variabel-variabel tersendiri.

- **Object destructuring:** `const { nama, umur } = mahasiswa` — sama dengan menulis `const nama = mahasiswa.nama;` secara terpisah.
- **Array destructuring:** `const [pertama, kedua] = arr` — mengambil elemen berdasarkan posisi.

Spread Operator (`...`) berfungsi untuk menyebarkan elemen array atau property object. Digunakan untuk menyalin, menggabungkan, atau mengoper data:

- Menyalin array: `const salinan = [...arrAsli]`
- Menggabungkan array: `const gabungan = [...arr1, ...arr2]`
- Menyalin object: `const obj2 = { ...obj1 }` (shallow copy)
- Menggabungkan/override: `const baru = { ...objLama, propertyBaru: 'nilai' }`

Rest Parameter (`...namaParam`) adalah kebalikan dari spread — ia mengumpulkan beberapa argumen menjadi satu array. Selalu ditempatkan di posisi terakhir.

3.2 Praktik: Destructuring, Spread, dan Rest

```
// =====  
// DESTRUCTURING, SPREAD OPERATOR, REST PARAMETER  
// =====  
  
// --- 1. Object Destructuring ---  
const produk = {  
  nama : 'Laptop Gaming',  
  merk : 'TechBrand',  
  harga : 12500000,  
  stok : 15,  
  garansi : '1 tahun',  
};  
  
const { nama, harga, stok } = produk;  
console.log('=== Object Destructuring ===');  
console.log(` ${nama} | Rp${harga.toLocaleString()} | Stok: ${stok}`);  
  
// Destructuring dengan rename (alias)  
const { nama: namaProduk, garansi: periodeGaransi } = produk;  
console.log(`Produk: ${namaProduk}, Garansi: ${periodeGaransi}`);  
  
// Destructuring dengan nilai default  
const { warna = 'Tidak diketahui', stok: jumlahStok = 0 } = produk;  
console.log(`Warna: ${warna} | Stok: ${jumlahStok}`);  
  
// --- 2. Array Destructuring ---
```

```

const koordinat = [10, 25, 5];
const [x, y, z] = koordinat;
console.log('\n=== Array Destructuring ===');
console.log(`x=${x}, y=${y}, z=${z}`);

// Skip elemen dengan koma
const [, kedua, ] = [100, 200, 300, 400];
console.log('Elemen kedua:', kedua);

// Swap dua variabel
let a = 'pertama', b = 'kedua';
console.log('Sebelum swap:', a, b);
[a, b] = [b, a];
console.log('Setelah swap:', a, b);

// --- 3. Spread Operator ---
const buah = ['Apel', 'Mangga', 'Jeruk'];
const sayur = ['Bayam', 'Wortel'];
const salinanBuah = [...buah];
salinanBuah.push('Pisang');

console.log('\n=== Spread Operator ===');
console.log('Asli :', buah);
console.log('Salinan:', salinanBuah);

const semuaMakanan = [...buah, ...sayur, 'Tempe'];
console.log('Gabungan:', semuaMakanan);

const profil = { nama: 'Siti', umur: 22, kota: 'Jakarta' };
const profilUpdate = { ...profil, kota: 'Bandung', pekerjaan: 'Developer' };
console.log('Profil asli :', profil);
console.log('Profil update:', profilUpdate);

// --- 4. Rest Parameter ---
function jumlahkanSemua(...angka) {
  console.log('Argumen diterima:', angka);
  return angka.reduce((total, n) => total + n, 0);
}

console.log('\n=== Rest Parameter ===');
console.log('Total:', jumlahkanSemua(1, 2, 3));
console.log('Total:', jumlahkanSemua(10, 20, 30, 40, 50));

// Rest di destructuring: ambil sisanya
const [kepala, ...ekor] = [1, 2, 3, 4, 5];
console.log('Kepala:', kepala);
console.log('Ekor :', ekor);

```

Analisis: Spread operator `[...buah]` membuat salinan array yang independen — menambahkan elemen ke `salinanBuah` tidak mempengaruhi `buah` asli. Pada `profilUpdate`, property `kota` di-override menjadi 'Bandung', sementara property lain diwarisi dari `profil`.

Laporan Praktikum Modul 4 — Object, Class & OOP

3.3 Latihan 2: Pengolahan Data dengan Destructuring & Spread

```

// =====
// LATIHAN 2: PENGOLAHAN DATA DENGAN DESTRUCTURING & SPREAD
// =====

// --- Langkah 2: Gabungkan tim ---
const timA = ['Budi', 'Siti', 'Ahmad'];
const timB = ['Rina', 'Doni'];

```



```
// Spread untuk menggabungkan + tambah anggota baru
const timGabungan = [...timA, ...timB, 'Zahra'];
console.log(`\n=== Tim Gabungan ===`);
console.log('Anggota:', timGabungan);

// Destructuring: kapten dan sisanya
const [kapten, ...anggota] = timGabungan;
console.log(`Kapten : ${kapten}`);
console.log(`Anggota : ${anggota.join(', ')}`);

// --- Langkah 5-7: Pengaturan dengan spread ---
const pengaturanDefault = {
  tema      : 'terang',
  bahasa    : 'id',
  notifikasi : true,
  fontSize  : 14
};

// Spread: warisi default, override tema dan fontSize
const pengaturanUser = {
  ...pengaturanDefault,
  tema      : 'gelap',
  fontSize  : 16
};

console.log(`\n=== Pengaturan User ===`);
console.log('Pengaturan lengkap:', pengaturanUser);

// Destructuring: ambil tema dan fontSize
const { tema, fontSize } = pengaturanUser;
console.log(`Tema      : ${tema}`);
console.log(`Font Size: ${fontSize}px`);
```

Output Latihan 2

```
=== Tim Gabungan ===
Anggota: [ 'Budi', 'Siti', 'Ahmad', 'Rina', 'Doni', 'Zahra' ]
Kapten  : Budi
Anggota : Siti, Ahmad, Rina, Doni, Zahra

=== Pengaturan User ===
Pengaturan lengkap: { tema: 'gelap', bahasa: 'id', notifikasi: true, fontSize: 16 }
Tema      : gelap
Font Size: 16px
```

Laporan Praktikum Modul 4 — Object, Class & OOP

BAB 4 — Class

4.1 Teori: Object-Oriented Programming dengan Class

Class adalah sebuah *blueprint* (cetak biru) untuk membuat object. Seperti cetakan kue — dari satu cetakan, kamu bisa membuat banyak kue dengan bentuk yang sama tapi isian berbeda.

Class

Blueprint/template untuk membuat object.

Instance	Object yang dibuat dari sebuah class (menggunakan <code>new</code>).
Constructor	Method spesial yang dipanggil otomatis saat <code>new ClassName()</code> dieksekusi.
Property	Data yang dimiliki setiap instance (didefinisikan dengan <code>this.prop</code> di constructor).
Method	Function yang dimiliki setiap instance, didefinisikan di dalam body class.
this	Merujuk ke instance yang sedang memanggil method.

4 Prinsip Utama OOP

1. **Encapsulation** — menyatukan data dan method dalam satu unit (class).
2. **Inheritance** — class baru bisa mewarisi property & method dari class lain.
3. **Abstraction** — menyembunyikan detail implementasi, hanya tampilkan yang diperlukan.
4. **Polymorphism** — method dengan nama sama bisa memiliki implementasi berbeda.

4.2 Praktik: Mendefinisikan Class

```
// =====
// CLASS: CONSTRUCTOR, PROPERTY, METHOD
// =====

// --- 1. Mendefinisikan class ---
class Mahasiswa {
  constructor(nama, nim, jurusan) {
    this.nama = nama;
    this.nim = nim;
    this.jurusan = jurusan;
    this.nilai = [];
  }

  tambahNilai(nilai) {
    this.nilai.push(nilai);
  }

  hitungRataRata() {
    if (this.nilai.length === 0) return 0;
    const total = this.nilai.reduce((sum, n) => sum + n, 0);
    return (total / this.nilai.length).toFixed(2);
  }

  getInfo() {
    return `[${this.nim}] ${this.nama} - ${this.jurusan} | Rata-rata: ${this.hitungRataRata()}`;
  }
}

// --- 2. Membuat instance ---
const mhs1 = new Mahasiswa('Budi Santoso', '2021001', 'Informatika');
const mhs2 = new Mahasiswa('Siti Rahayu', '2021002', 'Sistem Informasi');
const mhs3 = new Mahasiswa('Ahmad Fauzi', '2021003', 'Informatika');

mhs1.tambahNilai(85); mhs1.tambahNilai(90); mhs1.tambahNilai(78);
```

```
mhs2.tambahNilai(92); mhs2.tambahNilai(88); mhs2.tambahNilai(95);  
mhs3.tambahNilai(70); mhs3.tambahNilai(75);
```

```
console.log('=== Data Mahasiswa ===');  
console.log(mhs1.getInfo());  
console.log(mhs2.getInfo());  
console.log(mhs3.getInfo());
```

```
// --- 3. Static method ---
```

```
class Kalkulator {  
    static tambah(a, b) { return a + b; }  
    static kurang(a, b) { return a - b; }  
    static kali(a, b) { return a * b; }  
    static bagi(a, b) {  
        if (b === 0) return 'Error: bagi nol';  
        return a / b;  
    }  
}
```

```
console.log("\n=== Static Method ===");  
console.log('10 + 5 =', Kalkulator.tambah(10, 5));  
console.log('10 * 5 =', Kalkulator.kali(10, 5));  
console.log('10 / 0 =', Kalkulator.bagi(10, 0));
```

```
// --- 4. Getter dan Setter ---
```

```
class Lingkaran {  
    constructor(jariJari) {  
        this._jariJari = jariJari;  
    }  
    get jariJari() { return this._jariJari; }  
    get luas() { return (Math.PI * this._jariJari ** 2).toFixed(2); }  
    get keliling() { return (2 * Math.PI * this._jariJari).toFixed(2); }  
    set jariJari(nilai) {  
        if (nilai <= 0) throw new Error('Jari-jari harus positif!');  
        this._jariJari = nilai;  
    }  
}
```

```
console.log("\n=== Getter & Setter ===");  
const l = new Lingkaran(7);  
console.log('Jari-jari:', l.jariJari);  
console.log('Luas    :', l.luas);  
console.log('Keliling:', l.keliling);  
l.jariJari = 10;  
console.log('Setelah diubah - Luas:', l.luas);
```

Output

```
=== Data Mahasiswa ===  
[2021001] Budi Santoso - Informatika | Rata-rata: 84.33  
[2021002] Siti Rahayu - Sistem Informasi | Rata-rata: 91.67  
[2021003] Ahmad Fauzi - Informatika | Rata-rata: 72.50  
  
=== Static Method ===  
10 + 5 = 15  
10 * 5 = 50  
10 / 0 = Error: bagi nol  
  
=== Getter & Setter ===  
Jari-jari: 7  
Luas    : 153.94  
Keliling : 43.98  
Setelah diubah - Luas: 314.16
```

4.3 Latihan 3: Class BankAccount

```
// =====
// LATIHAN 3: CLASS BANK ACCOUNT
// =====

class BankAccount {
  constructor(namaPemilik, saldoAwal = 0) {
    this.namaPemilik = namaPemilik;
    this.saldo = saldoAwal;
  }

  // Menambah saldo
  setor(jumlah) {
    if (jumlah <= 0) {
      console.log('[ERROR] Jumlah setor harus lebih dari 0.');
```

return;

```
    }
    this.saldo += jumlah;
    console.log('[SETOR] ${this.namaPemilik} menyetor Rp${jumlah.toLocaleString()}. Saldo: Rp${this.saldo.toLocaleString()}');
```

}

```
  // Mengurangi saldo
  tarik(jumlah) {
    if (jumlah <= 0) {
      console.log('[ERROR] Jumlah tarik harus lebih dari 0.');
```

return;

```
    }
    if (jumlah > this.saldo) {
      console.log('[ERROR] Saldo tidak mencukupi. Saldo saat ini: Rp${this.saldo.toLocaleString()}');
```

return;

```
    }
    this.saldo -= jumlah;
    console.log('[TARIK] ${this.namaPemilik} menarik Rp${jumlah.toLocaleString()}. Saldo: Rp${this.saldo.toLocaleString()}');
```

}

```
  // Cek saldo
  cekSaldo() {
    console.log(`Pemilik: ${this.namaPemilik} | Saldo: Rp${this.saldo.toLocaleString()}`);
  }

  // Static method: transfer antar akun
  static transfer(akun1, akun2, jumlah) {
    console.log(`\n--- Transfer Rp${jumlah.toLocaleString()} dari ${akun1.namaPemilik} ke ${akun2.namaPemilik} ---`);
    akun1.tarik(jumlah);
    akun2.setor(jumlah);
  }
}

// --- Uji Coba ---
console.log(`\n=== Uji Coba BankAccount ===`);
const akunBudi = new BankAccount('Budi', 500000);
const akunSiti = new BankAccount('Siti', 300000);

akunBudi.cekSaldo();
akunSiti.cekSaldo();

// Kasus normal
akunBudi.setor(200000);
```

```

akunBudi.tarik(100000);

// Kasus error: saldo tidak cukup
akunBudi.tarik(999999);

// Kasus error: jumlah negatif
akunBudi.setor(-50000);
akunBudi.tarik(0);

// Transfer
BankAccount.transfer(akunBudi, akunSiti, 150000);
akunBudi.cekSaldo();
akunSiti.cekSaldo();

// Transfer gagal: saldo tidak cukup
BankAccount.transfer(akunSiti, akunBudi, 1000000);

```

Output Latihan 3

```

=== Uji Coba BankAccount ===
Pemilik: Budi | Saldo: Rp500.000
Pemilik: Siti | Saldo: Rp300.000
[SETOR] Budi menyetor Rp200.000. Saldo: Rp700.000
[TARIK] Budi menarik Rp100.000. Saldo: Rp600.000
[ERROR] Saldo tidak mencukupi. Saldo saat ini: Rp600.000
[ERROR] Jumlah setor harus lebih dari 0.
[ERROR] Jumlah tarik harus lebih dari 0.

--- Transfer Rp150.000 dari Budi ke Siti ---
[TARIK] Budi menarik Rp150.000. Saldo: Rp450.000
[SETOR] Siti menyetor Rp150.000. Saldo: Rp450.000
Pemilik: Budi | Saldo: Rp450.000
Pemilik: Siti | Saldo: Rp450.000

--- Transfer Rp1.000.000 dari Siti ke Budi ---
[ERROR] Saldo tidak mencukupi. Saldo saat ini: Rp450.000
[ERROR] Jumlah setor harus lebih dari 0.

```

Laporan Praktikum Modul 4 — Object, Class & OOP

BAB 5 — Inheritance (Pewarisan)

5.1 Teori: Pewarisan Class

Inheritance memungkinkan sebuah class baru (*child/subclass*) untuk mewarisi semua property dan method dari class lain (*parent/superclass*). Child class kemudian bisa menambahkan property/method baru atau menimpa (*override*) method yang sudah ada.

`extends`

Mendeklarasikan bahwa sebuah class mewarisi dari class lain.

`super()`

Memanggil constructor milik parent class. Wajib dipanggil sebelum `this`.

`super.namaMethod()`

Memanggil method milik parent class dari dalam child class.

Kapan menggunakan inheritance? Gunakan ketika ada hubungan "*adalah sebuah*" (is-a): Kucing IS-A Hewan, Mobil IS-A Kendaraan, Mahasiswa IS-A Orang.

5.2 Praktik: Inheritance dan Polymorphism

```
// =====  
// INHERITANCE: EXTENDS, SUPER, OVERRIDE  
// =====  
  
// --- Parent Class ---  
class Kendaraan {  
  constructor(merk, model, tahun) {  
    this.merk = merk;  
    this.model = model;  
    this.tahun = tahun;  
    this.kecepatanSaatIni = 0;  
  }  
  akselerasi(tambahan) {  
    this.kecepatanSaatIni += tambahan;  
    console.log(`${this.merk} ${this.model}: kecepatan ${this.kecepatanSaatIni} km/h`);  
  }  
  rem() {  
    this.kecepatanSaatIni = 0;  
    console.log(`${this.merk} ${this.model}: berhenti.`);  
  }  
  info() {  
    return `${this.tahun} ${this.merk} ${this.model}`;  
  }  
}  
  
// --- Child Class: Mobil ---  
class Mobil extends Kendaraan {  
  constructor(merk, model, tahun, jumlahPintu) {  
    super(merk, model, tahun);  
    this.jumlahPintu = jumlahPintu;  
  }  
  bunyikanKlakson() {  
    console.log(`${this.merk}: Beep beep!`);  
  }  
  info() {  
    return `${super.info()} (${this.jumlahPintu} pintu)`;  
  }  
}  
  
// --- Child Class: Motor ---  
class Motor extends Kendaraan {  
  constructor(merk, model, tahun, jenisMotor) {  
    super(merk, model, tahun);  
    this.jenisMotor = jenisMotor;  
  }  
  wheelie() {  
    if (this.kecepatanSaatIni > 50) {  
      console.log(`${this.merk}: Wheelie!`);  
    } else {  
      console.log('Kecepatan terlalu rendah untuk wheelie.');    }  
  }  
  info() {  
    return `${super.info()} [${this.jenisMotor}]`;  
  }  
}  
  
// --- Demonstrasi ---  
const mobil = new Mobil('Toyota', 'Avanza', 2022, 4);  
const motor = new Motor('Honda', 'CBR600RR', 2021, 'Sport');
```

```

console.log('=== Info Kendaraan ===');
console.log(mobil.info());
console.log(motor.info());

console.log('\n=== Aksi ===');
mobil.akselerasi(60);
mobil.bunyikanKlakson();
motor.akselerasi(80);
motor.wheelie();
motor.rem();

console.log('\n=== instanceof ===');
console.log('mobil instanceof Mobil :', mobil instanceof Mobil);
console.log('mobil instanceof Kendaraan:', mobil instanceof Kendaraan);
console.log('motor instanceof Mobil :', motor instanceof Mobil);

console.log('\n=== Polymorphism ===');
const semuaKendaraan = [mobil, motor];
semuaKendaraan.forEach(k => console.log(k.info()));

```

Output

```

=== Info Kendaraan ===
2022 Toyota Avanza (4 pintu)
2021 Honda CBR600RR [Sport]

=== Aksi ===
Toyota Avanza: kecepatan 60 km/h
Toyota: Beep beep!
Honda CBR600RR: kecepatan 80 km/h
Honda: Wheelie!
Honda CBR600RR: berhenti.

=== instanceof ===
mobil instanceof Mobil : true
mobil instanceof Kendaraan : true
motor instanceof Mobil : false

=== Polymorphism ===
2022 Toyota Avanza (4 pintu)
2021 Honda CBR600RR [Sport]

```

Laporan Praktikum Modul 4 — Object, Class & OOP

5.3 Latihan 4: Hierarki Class Hewan

```

// =====
// LATIHAN 4: HIERARKI CLASS HEWAN
// =====

// --- Parent Class: Hewan ---
class Hewan {
  constructor(nama, suara) {
    this.nama = nama;
    this.suara = suara;
  }
  bersuara() {
    console.log(`${this.nama} berkata: ${this.suara}`);
  }
  info() {
    console.log(`${this.nama} — Suara: ${this.suara}`);
  }
}

```

```
// --- Child Class: Anjing ---
class Anjing extends Hewan {
  constructor(nama, suara) {
    super(nama, suara);
  }
  fetch() {
    console.log(`${this.nama} mengambil bola!`);
  }
  info() {
    console.log(`${this.nama} — Suara: ${this.suara} (jenis: anjing)`);
  }
}

// --- Child Class: Kucing ---
class Kucing extends Hewan {
  constructor(nama, suara) {
    super(nama, suara);
  }
  cakarSofa() {
    console.log(`${this.nama} mencakar sofa!`);
  }
  bersuara() {
    super.bersuara();
    console.log("Purrr...");
  }
}

// --- Instance ---
const anjing1 = new Anjing('Buddy', 'Guk guk!');
const anjing2 = new Anjing('Rex', 'Woof!');
const kucing1 = new Kucing('Kitty', 'Meong!');
const kucing2 = new Kucing('Tom', 'Nyaa~');

// --- Polymorphism: satu array, perilaku berbeda ---
console.log(`\n=== Demonstrasi Polymorphism Hewan ===`);
const semuaHewan = [anjing1, anjing2, kucing1, kucing2];
semuaHewan.forEach(h => {
  h.bersuara();
  h.info();
  console.log('---');
});

// Demo method unik
console.log(`\n=== Method Unik ===`);
anjing1.fetch();
kucing1.cakarSofa();
```

Output Latihan 4

```
=== Demonstrasi Polymorphism Hewan ===
Buddy berkata: Guk guk!
Buddy — Suara: Guk guk! (jenis: anjing)
---
Rex berkata: Woof!
Rex — Suara: Woof! (jenis: anjing)
---
Kitty berkata: Meong!
Purrr...
Kitty — Suara: Meong! (jenis: kucing)
---
Tom berkata: Nyaa~
Purrr...
```



```
Tom — Suara: Nyaa~ (jenis: kucing)
---

=== Method Unik ===
Buddy mengambil bola!
Kitty mencakar sofa!
```

BAB 6 — Implementasi Stack dan Queue

6.1 Teori: Stack dan Queue — Jembatan ke DSA

Ini adalah bagian paling menarik — kita akan menggunakan semua yang telah dipelajari (Class, array, method) untuk mengimplementasikan dua struktur data fundamental.

Prinsip	LIFO (Last In, First Out)	FIFO (First In, First Out)
Analogi	Tumpukan piring	Antrian kasir
Tambah	<code>push(data)</code>	<code>enqueue(data)</code>
Ambil	<code>pop()</code>	<code>dequeue()</code>
Lihat atas/depan	<code>peek()</code>	<code>front()</code>
Penggunaan nyata	Undo/Redo, call stack JS, navigasi browser	Print queue, buffer streaming, antrian tugas OS

6.2 Praktik: Implementasi Stack dan Queue

```
// =====
// IMPLEMENTASI STACK DAN QUEUE MENGGUNAKAN CLASS
// =====

// ===== CLASS STACK — prinsip LIFO =====
class Stack {
  constructor() {
    this.data = [];
    this.ukuran = 0;
  }
  push(elemen) {
    this.data.push(elemen);
    this.ukuran++;
    console.log(` Push '${elemen}' → Stack: [${this.data}]`);
  }
  pop() {
    if (this.isEmpty()) {
      console.log(' Stack kosong! Tidak bisa pop. ');
      return null;
    }
    const dikeluarkan = this.data.pop();
    this.ukuran--;
    console.log(` Pop '${dikeluarkan}' → Stack: [${this.data}]`);
    return dikeluarkan;
  }
}
```

```

    peek() {
        if (this.isEmpty()) return null;
        return this.data[this.data.length - 1];
    }
    isEmpty() { return this.ukuran === 0; }
    size() { return this.ukuran; }
    tampilkan() {
        console.log(` Stack (atas→bawah): [${[...this.data].reverse()}]`);
    }
}

// ===== CLASS QUEUE — prinsip FIFO =====
class Queue {
    constructor() {
        this.data = [];
        this.ukuran = 0;
    }
    enqueue(elemen) {
        this.data.push(elemen);
        this.ukuran++;
        console.log(` Enqueue '${elemen}' → Queue: [${this.data}]`);
    }
    dequeue() {
        if (this.isEmpty()) {
            console.log(' Queue kosong! Tidak bisa dequeue. ');
            return null;
        }
        const dikeluarkan = this.data.shift();
        this.ukuran--;
        console.log(` Dequeue '${dikeluarkan}' → Queue: [${this.data}]`);
        return dikeluarkan;
    }
    front() {
        if (this.isEmpty()) return null;
        return this.data[0];
    }
    isEmpty() { return this.ukuran === 0; }
    size() { return this.ukuran; }
    tampilkan() {
        console.log(` Queue (depan→belakang): [${this.data}]`);
    }
}

// ===== DEMONSTRASI STACK =====
console.log('=====');
console.log('DEMONSTRASI STACK (LIFO)');
console.log('=====');

const stack = new Stack();
stack.push('A');
stack.push('B');
stack.push('C');
console.log(' Peek (puncak):', stack.peak());
stack.pop();
stack.pop();
stack.pop();
stack.pop();

// Simulasi Undo
console.log(`\n--- Simulasi Undo ---`);
const riwayatAksi = new Stack();
riwayatAksi.push('Ketik: Halo');
riwayatAksi.push('Ketik: Dunia');

```

```

riwayatAksi.push('Bold teks');
riwayatAksi.push('Ubah warna');
console.log(' Undo:', riwayatAksi.pop());
console.log(' Undo:', riwayatAksi.pop());
riwayatAksi.tampilkan();

// ===== DEMONSTRASI QUEUE =====
console.log("\n=====");
console.log('DEMONSTRASI QUEUE (FIFO)');
console.log('=====');

const queue = new Queue();
queue.enqueue('X');
queue.enqueue('Y');
queue.enqueue('Z');
console.log(' Front (depan):', queue.front());
queue.dequeue();
queue.dequeue();
queue.dequeue();
queue.dequeue();

// Simulasi Antrian Printer
console.log("\n--- Simulasi Antrian Printer ---");
const antrianPrinter = new Queue();
antrianPrinter.enqueue('Dokumen-Laporan.pdf');
antrianPrinter.enqueue('Foto-Wisuda.jpg');
antrianPrinter.enqueue('Tugas-Akhir.docx');
console.log(' Mencetak...');
while (!antrianPrinter.isEmpty()) {
  const dokumen = antrianPrinter.dequeue();
  console.log(` Selesai mencetak: ${dokumen}`);
}
console.log(' Semua dokumen selesai dicetak.');
```

Output

```

=====
DEMONSTRASI STACK (LIFO)
=====
Push 'A' → Stack: [A]
Push 'B' → Stack: [A,B]
Push 'C' → Stack: [A,B,C]
Peek (puncak): C
Pop 'C' → Stack: [A,B]
Pop 'B' → Stack: [A]
Pop 'A' → Stack: []
Stack kosong! Tidak bisa pop.

--- Simulasi Undo ---
Push 'Ketik: Halo' → Stack: [Ketik: Halo]
Push 'Ketik: Dunia' → Stack: [Ketik: Halo,Ketik: Dunia]
Push 'Bold teks' → Stack: [Ketik: Halo,Ketik: Dunia,Bold teks]
Push 'Ubah warna' → Stack: [Ketik: Halo,Ketik: Dunia,Bold teks,Ubah warna]
Undo: Ubah warna
Undo: Bold teks
Stack (atas→bawah): [Ketik: Dunia,Ketik: Halo]

=====
DEMONSTRASI QUEUE (FIFO)
=====
Enqueue 'X' → Queue: [X]
Enqueue 'Y' → Queue: [X,Y]
Enqueue 'Z' → Queue: [X,Y,Z]
```

```
Front (depan): X
Dequeue 'X' → Queue: [Y,Z]
Dequeue 'Y' → Queue: [Z]
Dequeue 'Z' → Queue: []
Queue kosong! Tidak bisa dequeue.
```

--- Simulasi Antrian Printer ---

```
Enqueue 'Dokumen-Laporan.pdf' → Queue: [Dokumen-Laporan.pdf]
Enqueue 'Foto-Wisuda.jpg' → Queue: [Dokumen-Laporan.pdf,Foto-Wisuda.jpg]
Enqueue 'Tugas-Akhir.docx' → Queue: [Dokumen-Laporan.pdf,Foto-Wisuda.jpg,Tugas-Akhir.docx]
Mencetak...
Dequeue 'Dokumen-Laporan.pdf' → Queue: [Foto-Wisuda.jpg,Tugas-Akhir.docx]
Selesai mencetak: Dokumen-Laporan.pdf
Dequeue 'Foto-Wisuda.jpg' → Queue: [Tugas-Akhir.docx]
Selesai mencetak: Foto-Wisuda.jpg
Dequeue 'Tugas-Akhir.docx' → Queue: []
Selesai mencetak: Tugas-Akhir.docx
Semua dokumen selesai dicetak.
```

Analisis: Pada Stack, C dimasukkan terakhir tapi keluar pertama (LIFO). Pada Queue, X dimasukkan pertama dan keluar pertama (FIFO). Simulasi Undo menunjukkan aksi terbaru yang di-undo terlebih dahulu — persis seperti Ctrl+Z di aplikasi nyata.

Laporan Praktikum Modul 4 — Object, Class & OOP

BAB 7 — Tugas Praktikum

7.1 Tugas 1: Sistem Manajemen Toko Online (OOP)

Tugas ini mengimplementasikan sistem manajemen toko online menggunakan konsep OOP lengkap: inheritance, method overriding, polymorphism, serta penggunaan array methods tingkat lanjut.

```
// =====
// TUGAS 1: SISTEM MANAJEMEN TOKO ONLINE (OOP)
// =====

// --- Parent Class: Produk ---
class Produk {
  constructor(id, nama, harga, stok) {
    this.id = id;
    this.nama = nama;
    this.harga = harga;
    this.stok = stok;
  }

  info() {
    console.log(`[${this.id}] ${this.nama} | Harga: Rp${this.harga.toLocaleString()} | Stok: ${this.stok}`);
  }

  tersedia() {
    return this.stok > 0;
  }

  jual(jumlah) {
    if (jumlah <= 0) {
      console.log(`[ERROR] Jumlah jual harus lebih dari 0.`);
      return false;
    }
    if (jumlah > this.stok) {
```

```

        console.log(`[ERROR] Stok ${this.nama} tidak mencukupi (sisa: ${this.stok}).`);
        return false;
    }
    this.stok -= jumlah;
    console.log(`[SUKSES] Terjual ${jumlah}x ${this.nama}. Stok sisa: ${this.stok}`);
    return true;
}
}

// --- Child Class: ProdukDigital ---
class ProdukDigital extends Produk {
    constructor(id, nama, harga, stok, ukuranFile, formatFile) {
        super(id, nama, harga, stok);
        this.ukuranFile = ukuranFile;
        this.formatFile = formatFile;
    }

    info() {
        console.log(`[${this.id}] ${this.nama} (Digital) | Harga: Rp${this.harga.toLocaleString()} | Ukuran: ${this.ukuranFile}MB |
Format: ${this.formatFile}`);
    }

    jual(jumlah) {
        // Produk digital tidak mengurangi stok fisik
        console.log(`[DIGITAL] Terjual ${jumlah}x "${this.nama}" — link download dikirim.`);
        return true;
    }

    download() {
        console.log(`[DOWNLOAD] Mengunduh "${this.nama}" (${this.ukuranFile}MB, ${this.formatFile})...`);
    }
}

// --- Child Class: ProdukFisik ---
class ProdukFisik extends Produk {
    constructor(id, nama, harga, stok, beratGram, dimensi) {
        super(id, nama, harga, stok);
        this.beratGram = beratGram;
        this.dimensi = dimensi;
    }

    info() {
        console.log(`[${this.id}] ${this.nama} (Fisik) | Harga: Rp${this.harga.toLocaleString()} | Berat: ${this.beratGram}g |
Dimensi: ${this.dimensi} | Stok: ${this.stok}`);
    }

    hitungOngkir(tarifPerKg) {
        const beratKg = this.beratGram / 1000;
        const ongkir = Math.ceil(beratKg * tarifPerKg);
        return ongkir;
    }
}

// --- Buat Instance ---
const daftarProduk = [
    new ProdukDigital('D001', 'E-Book Algoritma', 75000, 999, 15, 'PDF'),
    new ProdukDigital('D002', 'Kursus Video JavaScript', 250000, 999, 1200, 'MP4'),
    new ProdukFisik('F001', 'Laptop Gaming ROG', 18000000, 5, 2400, '36x26x3 cm'),
    new ProdukFisik('F002', 'Mouse Wireless Logitech', 350000, 20, 120, '10x6x4 cm'),
    new ProdukFisik('F003', 'Keyboard Mechanical', 850000, 0, 900, '45x15x4 cm'),
];

// (a) forEach: tampilkan semua info produk

```

```

console.log('=== (a) Semua Produk ===');
daftarProduk.forEach(p => p.info());

// (b) filter: produk yang tersedia
console.log('\n=== (b) Produk Tersedia ===');
const produkTersedia = daftarProduk.filter(p => p.tersedia());
produkTersedia.forEach(p => console.log(` - ${p.nama}`));

// (c) map: array nama produk saja
console.log('\n=== (c) Daftar Nama Produk ===');
const namaProduk = daftarProduk.map(p => p.nama);
console.log(namaProduk);

// --- Demo tambahan ---
console.log('\n=== Demo Penjualan ===');
daftarProduk[0].jual(1);
daftarProduk[0].download();

daftarProduk[2].jual(2);
const ongkirLaptop = daftarProduk[2].hitungOngkir(15000);
console.log(`Ongkir Laptop: Rp${ongkirLaptop.toLocaleString()}`);

daftarProduk[4].jual(1); // stok 0, harus gagal

```

Output Tugas 1

```

=== (a) Semua Produk ===
[D001] E-Book Algoritma (Digital) | Harga: Rp75.000 | Ukuran: 15MB | Format: PDF
[D002] Kursus Video JavaScript (Digital) | Harga: Rp250.000 | Ukuran: 1200MB | Format: MP4
[F001] Laptop Gaming ROG (Fisik) | Harga: Rp18.000.000 | Berat: 2400g | Dimensi: 36x26x3 cm | Stok: 5
[F002] Mouse Wireless Logitech (Fisik) | Harga: Rp350.000 | Berat: 120g | Dimensi: 10x6x4 cm | Stok: 20
[F003] Keyboard Mechanical (Fisik) | Harga: Rp850.000 | Berat: 900g | Dimensi: 45x15x4 cm | Stok: 0

=== (b) Produk Tersedia ===
- E-Book Algoritma
- Kursus Video JavaScript
- Laptop Gaming ROG
- Mouse Wireless Logitech

=== (c) Daftar Nama Produk ===
[ 'E-Book Algoritma', 'Kursus Video JavaScript', 'Laptop Gaming ROG', 'Mouse Wireless Logitech', 'Keyboard Mechanical' ]

=== Demo Penjualan ===
[DIGITAL] Terjual 1x "E-Book Algoritma" — link download dikirim.
[DOWNLOAD] Mengunduh "E-Book Algoritma" (15MB, PDF)...
[SUKSES] Terjual 2x Laptop Gaming ROG. Stok sisa: 3
Ongkir Laptop: Rp36.000
[ERROR] Stok Keyboard Mechanical tidak mencukupi (sisa: 0).

```

Laporan Praktikum Modul 4 — Object, Class & OOP

7.2 Tugas 2: Stack Kalkulator (Evaluasi Ekspresi)

Tugas ini menggunakan class Stack untuk mengecek apakah tanda kurung dalam sebuah ekspresi matematika seimbang (balanced parentheses). Ini adalah salah satu aplikasi klasik Stack dalam ilmu komputer.

```

// =====
// TUGAS 2: STACK KALKULATOR — CEK KURUNG SEIMBANG
// =====

// --- Class Stack (disalin dari 05-stack-queue.js) ---

```

```

class Stack {
  constructor() {
    this.data = [];
    this.ukuran = 0;
  }
  push(elemen) {
    this.data.push(elemen);
    this.ukuran++;
  }
  pop() {
    if (this.isEmpty()) return null;
    this.ukuran--;
    return this.data.pop();
  }
  peek() {
    if (this.isEmpty()) return null;
    return this.data[this.data.length - 1];
  }
  isEmpty() { return this.ukuran === 0; }
  size() { return this.ukuran; }
}

// --- Function: Cek Kurung Seimbang ---
function cekKurungSeimbang(ekspresi) {
  const stack = new Stack();

  for (let i = 0; i < ekspresi.length; i++) {
    const char = ekspresi[i];

    if (char === '(') {
      // Jika kurung buka, push ke stack
      stack.push(char);
    } else if (char === ')') {
      // Jika kurung tutup, coba pop dari stack
      if (stack.isEmpty()) {
        // Stack kosong saat pop → tidak seimbang
        return false;
      }
      stack.pop();
    }
    // Karakter lain diabaikan
  }

  // Di akhir, stack harus kosong agar seimbang
  return stack.isEmpty();
}

// --- Pengujian ---
console.log('=====');
console.log('CEK KURUNG SEIMBANG');
console.log('=====');

const testCases = [
  '(2 + 3) * (4 - 1)',
  '((a + b)',
  ')(',
  '((( )))',
  '(a * (b + c)) - (d / (e + f))',
  '((( ))',
  '2 + 3 * 4', // tanpa kurung → seimbang
  '()()()', // beberapa pasang
  '()()()()', // nested + berturut
  '()())(', // tidak seimbang

```

```

];

testCases.forEach(ekspresi => {
  const hasil = cekKurungSeimbang(ekspresi);
  console.log(`${ekspresi} → Seimbang: ${hasil}`);
});

console.log("\n=====");
console.log('PENJELASAN LOGIKA:');
console.log('=====');
console.log('1. Iterasi setiap karakter dalam ekspresi. ');
console.log('2. Jika ketemu "(" → push ke stack. ');
console.log('3. Jika ketemu ")" → pop dari stack. ');
console.log('   - Jika stack kosong saat pop → TIDAK seimbang. ');
console.log('4. Di akhir iterasi, cek stack. ');
console.log('   - Stack kosong → SEIMBANG. ');
console.log('   - Stack tidak kosong → TIDAK seimbang. ');

```

Output Tugas 2

```

=====
CEK KURUNG SEIMBANG
=====
'(2 + 3) * (4 - 1)' → Seimbang: true
'((a + b)' → Seimbang: false
')' → Seimbang: false
'((( )))' → Seimbang: true
'(a * (b + c)) - (d / (e + f))' → Seimbang: true
'((( )))' → Seimbang: false
'2 + 3 * 4' → Seimb

```